AI-Assisted Coding — Prompt and Response Log

Project: M&A Quant Optimizer

Subject: AI-Assisted Coding

This document presents the complete prompt-and-response history used in developing the M&A Quant Optimizer — a constrained multi-objective optimization system for M&A deal structuring. The record is divided into two parts: the planning phase conducted with ChatGPT, and the implementation phase conducted with Claude.

Part 1 covers nine exchanges in which the project concept, system architecture, UI specification, and requirements document were iteratively developed. Part 2 covers two exchanges in which the full-stack React application was generated and subsequently upgraded.

PART 1: Planning Phase — ChatGPT

The following exchanges document the planning and requirements-definition phase of the project, conducted using ChatGPT. Prompts have been reproduced in a clear and professional form while preserving the intent of each message. Responses have been cleaned of informal formatting while retaining all substantive technical and design content.

Exchange 1: Initial Scoping and Direction

PROMPT

I would like to develop a project for my AI-Assisted Coding subject using Claude and other free AI tools. My chosen topic is M&A Accretion/Dilution analysis. I am not deeply familiar with the financial domain, but I want to build something with genuine real-world impact that takes a unique approach. I would prefer to proceed step by step. Please ask me clarifying questions before we design the project.

RESPONSE — ChatGPT

This is a strong topic with significant potential if approached correctly. Most implementations stop at a basic EPS calculator, but this can be pushed into something far more interesting. Before designing the project, please answer the following questions:

1. Project Identity

Choose one primary identity for the project:

- A — Simple tool (calculator and basic UI)
- B — Smart system (AI-assisted decision support)
- C — Research-heavy model (prediction or optimization)
- D — Experimental and novel concept

2. Technical Level

- Low — frontend with basic logic
- Medium — some ML, backend, and APIs
- High — models, evaluation, and advanced system design

3. Primary Interest

- Finance understanding (learning M&A in depth)
- AI building (models, prompts, system design)
- Product building (UI/UX, website, users)

4. Constraints

Please share your deadline, whether this is a solo or team project, and whether you are comfortable using external APIs such as Claude or OpenAI.

Context on the Topic

Accretion/Dilution refers to the EPS impact of an M&A transaction: Company A acquires Company B, and the post-deal EPS either increases (accretive) or decreases (dilutive). Most student projects stop at computing this single metric. The opportunity lies in building on top of it.

Possible Directions

1. Explainable AI M&A Analyst — inputs financials, outputs accretive/dilutive verdict with risk factors and explanations.
2. What-If Simulation Engine — user adjusts deal structure, debt vs. equity, and synergy assumptions; system shows EPS impact dynamically and suggests the best deal structure.
3. AI-Designed Optimal Acquisition — user specifies a goal (maximize EPS or minimize dilution); AI suggests the deal structure to achieve it.
4. Real-World Data and Prediction — uses past M&A deals to predict whether a transaction was accretive. More ML-heavy, less product-oriented.

Exchange 2: Requirements and Constraints

PROMPT

My preferences are as follows. For project identity, I am leaning toward a combination of B and D — a smart, experimental system. The technical level should be high, though the implementation timeline is constrained to a single night. My primary interest is in combining finance and AI for real-world problem solving. This is a team project in which I am the sole contributor, so all tools should be free and accessible. For project direction, I prefer Option 2 (the what-if simulation engine), with Option 3 as a secondary interest. I am not interested in the explainable AI direction.

RESPONSE — ChatGPT

Given the one-night constraint, certain approaches would be high risk: training an ML model, building a full backend with authentication and a database, and attempting a real-time trading system are all inadvisable. Cloning a GitHub repository and modifying it is also risky as it leads to significant time loss navigating unfamiliar code.

The recommended approach is to build an AI-Powered M&A Deal Simulator and Optimizer. This is not a basic calculator and does not require heavy ML, but it presents as an advanced interactive system.

What the system does:

The user provides: Acquirer EPS, Target EPS, Deal Value, Cash vs. Stock percentage, and Synergy estimates. The system outputs: an accretive or dilutive verdict, an optimal deal structure recommendation, sensitivity analysis (how inputs affect the result), and an optional strategic explanation.

Why this approach works:

Real M&A analysis involves complex trade-offs across deal structure, financing mix, synergy assumptions, and valuation. Transforming this into an interactive decision system — rather than a static calculator — gives the project genuine utility and depth.

Recommended Architecture (Three Layers):

- Financial Engine — Core EPS computation in Python (or JavaScript).
- AI Optimization Layer — Claude API receives structured deal data and returns an improved deal structure with strategic justification.
- User Interface — Lovable.dev or Streamlit for rapid frontend generation.

Technology Stack:

- Frontend: Lovable.dev
- Backend: Python with FastAPI
- AI Integration: Claude API
- Hosting: Render or Streamlit Cloud (both free)

Core Calculation:

Post-deal EPS is computed as:

- New Income = Acquirer Income + Target Income + Synergy
- New Shares = Existing Shares + Newly Issued Shares (from stock-based financing)
- EPS_post = New Income / New Shares

Exchange 3: Technology Selection and Final Blueprint

PROMPT

The preferred stack is: Lovable for the frontend, the Claude API for AI integration. The system should be novel and accessible to real users.

RESPONSE — ChatGPT

The project is now fully scoped. The title to use is: AI-Assisted M&A Deal Optimization System for Accretion/Dilution Analysis.

System Architecture:

5. Financial Engine — computes post-deal EPS by combining acquirer and target income, adjusting share count based on the cash/stock financing mix, and comparing pre- and post-deal EPS.
6. AI Optimization Engine (Claude API) — receives the deal parameters and returns a structured suggestion: optimal cash-to-stock ratio, required synergy level, and strategic reasoning.
7. User Interface (Lovable) — accepts deal inputs and displays the calculation result, the AI-suggested optimized structure, and a side-by-side comparison.

System Flow:

User inputs deal parameters → Financial Engine computes EPS impact → Data is sent to Claude API → Claude returns an optimized deal structure → System recomputes using the suggested structure → Original vs. Optimized results are displayed side by side.

Claude API Prompt Template:

The following prompt structure is used to query Claude for deal optimization:

```
You are a financial M&A optimization expert.

Given the following deal parameters:
  - Acquirer EPS: {eps_a}
  - Target EPS: {eps_t}
  - Deal Value: {deal_value}
  - Cash Percentage: {cash_pct}
  - Stock Percentage: {stock_pct}
  - Estimated Synergies: {synergy}

Tasks:
  1. Determine if the deal is accretive or dilutive.
  2. Suggest an improved deal structure to maximize EPS.
  3. Recommend the optimal cash-to-stock ratio and required synergy level.
```

```
  4. Keep suggestions realistic and financially logical.

Output in structured JSON format:
{
  "status": "...",
  "improved_cash_pct": ...,
  "improved_stock_pct": ...,
  "required_synergy": ...,
  "explanation": "..."
}
```

Backend API Endpoints:

- POST /calculate — accepts EPS inputs, returns post-deal EPS result.
- POST /optimize — forwards structured deal data to Claude API, returns the suggested improved structure.

Exchange 4: System Enhancement Proposal

PROMPT

How can this project be made more compelling and technically impressive while remaining feasible within the available time?

RESPONSE — ChatGPT

The current version is solid but still fundamentally a single-query tool. The following three additions would significantly elevate the system without adding infeasible complexity:

1. Multi-Scenario Engine

Instead of computing a single result, generate multiple deal scenarios automatically: a base case, a high-synergy case, a low-synergy case, a high-cash/low-stock variant, and a high-stock/low-cash variant. The system then identifies and highlights the best-performing scenario. This shifts the system from providing a single answer to exploring a decision landscape.

2. Sensitivity Analysis

Rather than simply reporting EPS = X, the system should measure how much EPS changes when key inputs change. For example: if synergy increases by 10%, EPS increases by 0.5; if the stock percentage increases, dilution increases proportionally. This produces output that communicates cause-and-effect intelligence, not just a result.

3. AI as Strategist

Instead of asking the AI to return a single improved configuration, request 2 to 3 alternative deal strategies. Each strategy should explain its trade-offs — risk vs. return, dilution vs. control, and financing implications. This makes the AI layer feel like a genuine strategic advisor rather than a simple parameter tuner.

Revised System Output:

- Accretive/Dilutive verdict for the base case
- A scenario table showing 5 or more variations with EPS highlighted
- A sensitivity summary (e.g., 'EPS is most sensitive to synergy assumptions')
- An AI strategy panel showing 2 to 3 deal structures with explanations

Exchange 5: System Differentiation

PROMPT

I am uncertain about the system's novelty. Could a user not achieve the same result simply by querying Claude directly?

RESPONSE — ChatGPT

This is the right question to ask, and it is important to have a clear answer.

If a user queries Claude directly, the interaction is unstructured: the input has no guaranteed format, the computation is not deterministic, there is no scenario comparison, and there is no systematic sensitivity tracking. The output is generic advice.

This system is different in five specific ways:

8. Structured Input Enforcement — the system enforces valid financial inputs before any computation or AI call. Claude alone cannot enforce input schemas or financial constraints.
9. Deterministic Computation — EPS, sensitivity scores, and scenario rankings are computed mathematically by the system itself. The AI is only called after the computation is verified. Claude cannot reliably perform consistent arithmetic across repeated calls.
10. Multi-Scenario Framework — the system generates five or more deal configurations and ranks them by outcome. Claude alone would need explicit instruction to produce each variation, with no guarantee of consistency.
11. Optimization Loop — the system performs: Input → Compute → AI Suggest → Recompute → Compare. This structured feedback loop is a system design decision that Claude alone cannot replicate cleanly.
12. Sensitivity Intelligence — the system systematically measures how EPS changes when synergy or financing mix shifts by a defined increment. This requires deterministic parameter sweeps that LLMs do not perform on their own.

The system is best described as a hybrid intelligence system: it combines deterministic financial modeling with AI-assisted strategic reasoning. The analytical contribution is made by the financial engine; the AI layer provides interpretation and strategic framing of verified results.

Exchange 6: Search and Optimization Engine

PROMPT

The system still feels like a UI wrapper over an AI call. For it to be genuinely distinct, it should produce dynamic and realistic output across all possible scenarios, with a compelling user interface for each.

RESPONSE — ChatGPT

The correct upgrade is to shift the core idea from 'AI suggests a better deal' to 'Deterministic simulation engine with AI-guided search over the decision space.' This is what separates the system from a wrapper.

The Missing Layer: Search over the Decision Space

The system should programmatically generate 20 to 50 deal configurations by iterating over the full range of cash percentage (0 to 100) and synergy levels (low, base, high). For each configuration, it computes EPS and the sensitivity score. It then identifies the best-EPS scenario, the worst-case scenario, and the stable region (configurations with the lowest sensitivity). Only after this computation does the AI interpret the results.

This means the system is not asking the AI for an answer — it is computing the solution space, finding the optimal region, and then using the AI to explain the strategy. This is a decision intelligence system, not a query interface.

Dynamic UI Behaviors:

- Live simulation: moving the cash percentage slider updates the EPS chart in real time.
- Scenario table: all generated configurations are displayed with the best row highlighted.
- Stability indicator: a visual bar showing how sensitive the current configuration is to synergy variance.

System Positioning:

The system transforms accretion/dilution analysis into a simulation-driven optimization problem, augmented with AI for strategic interpretation of results.

Exchange 7: Project Requirements Document Request

PROMPT

Please provide complete project details — what to build, how to implement each section, and what to watch out for — formatted as a requirements document suitable for reference and for use when prompting Claude to generate code.

RESPONSE — ChatGPT

Project Title:

AI-Assisted Multi-Scenario Optimization System for M&A Accretion/Dilution Analysis

Objective:

To design a system that simulates multiple M&A deal structures, computes EPS impact across all scenarios, identifies optimal deal configurations, and uses AI to interpret and refine the financial strategies.

Problem with Existing Approaches:

- Traditional tools are static Excel-based models.
- They evaluate a single scenario at a time.
- They offer no optimization guidance.
- They provide no sensitivity insights.

System Architecture (Seven Modules):

13. Input Layer — User provides: Acquirer EPS, Target EPS, Deal Value, Cash %, Stock %, and Synergy estimate.
14. Financial Engine — Combines earnings, adjusts share count, computes post-deal EPS.
15. Scenario Generator — Iterates through Cash % (0 to 100) and Synergy levels (low, base, high) to produce 20 to 30 configurations.
16. Optimization Engine — From all scenarios, selects the maximum EPS configuration and identifies the worst-case outcome.
17. Sensitivity Analyzer — Measures how EPS changes when synergy is varied slightly. Outputs a classification such as 'high sensitivity' or 'low sensitivity.'
18. AI Layer — Claude API receives the base deal, the best scenario, and the worst scenario. It returns a structured JSON response containing improved deal structure and strategic reasoning.
19. UI Layer — Displays all results: the scenario table, the optimized configuration, AI suggestions, and comparison view.

Financial Model:

New Income = Acquirer Income + Target Income + Synergy

New Shares = Existing Shares + Newly Issued Shares

EPS_post = New Income / New Shares

Scenario Generation Logic:

- Cash %: [0, 25, 50, 75, 100]
- Synergy: [Low, Medium, High]
- Total combinations: 15 to 30

Optimization Logic:

From all generated scenarios, select the one with the maximum EPS. Compare it against the base case and report the percentage improvement.

Sensitivity Logic:

Vary synergy slightly (e.g., plus or minus 10%) and observe the resulting EPS change. If the change is large, classify the deal as high-sensitivity. If the change is small, classify as low-sensitivity.

AI Integration:

Send the base deal parameters, the best scenario, and the worst scenario to Claude. The system prompt instructs Claude to return a structured JSON array with an improved deal structure and a 2-sentence strategic justification for each Pareto-optimal configuration.

Implementation Guide:

20. Build the core EPS computation function. Watch for division by zero and unit consistency.
21. Build the scenario loop (20 to 30 combinations). Keep the iteration range realistic; too many combinations will affect performance.
22. Identify the maximum EPS configuration.
23. Add sensitivity measurement by varying synergy and comparing EPS outputs.
24. Integrate the Claude API — send a structured JSON payload, handle API errors, and parse the response JSON.
25. Build the UI in Lovable — inputs, scenario table, best-case highlight, and AI strategy cards.

Common Mistakes to Avoid:

- Overcomplicating the financial formulas — keep the model simple and deterministic.
- Adding too many features at once — build one module at a time.
- Relying entirely on the AI for computation — the AI layer should only interpret, not calculate.
- Skipping scenario comparison — the comparison view is central to the system's value.

Exchange 8: Detailed UI/UX Specification

PROMPT

The document lacks the visual and interaction detail needed. Please provide specific UI components, animations, and dynamic behaviors for each screen of the dashboard.

RESPONSE — ChatGPT

The system should feel like a deal command centre, not a form with a results panel. The user should be exploring a decision space, not submitting a query.

Screen 1: Deal Initialization Panel

Theme: dark finance-style background, subtle glow on inputs, glassmorphism cards.

Components: each input is presented as a card with micro-interactions. Acquirer EPS, Target EPS, and Deal Value use numerical input fields. Cash % and Synergy use range sliders. When the user adjusts a slider, a small pulse effect plays and the value increments smoothly. When all fields are valid, the Simulate button glows softly to indicate readiness.

Screen 2: Live Simulation Engine

Animation sequence on simulate: a loading line sweeps across the screen, the input cards fade out, and the results dashboard fades in. The chart draws progressively from left to right.

- EPS Trajectory Graph — X-axis: Cash %; Y-axis: EPS. The graph animates from left to right. The best-performing point glows; the worst-performing point is marked in a muted red.
- Scenario Table — rows appear with a staggered fade-in animation. The best row is highlighted with a soft green glow; the worst row with a faint red tint.
- Stability Indicator — a horizontal bar that fills based on the computed sensitivity score, with a tooltip explaining the sensitivity classification.
- Deal Verdict Card — shows Accretive or Dilutive with a percentage improvement. The card slides up on appearance and the status text renders with a typewriter effect.

Screen 3: AI Strategy Engine

Output format: not a chatbot response or a paragraph of text, but a set of structured strategy cards. Each card represents one deal configuration returned by the AI.

- Strategy card fields: deal label (e.g., Balanced Growth), Cash and Stock percentages, expected EPS with direction indicator, and a 2-sentence AI-generated insight.
- Cards appear one by one with a staggered entrance animation. Each card slightly lifts on hover with an intensified border glow. Clicking a card expands it to reveal a deeper explanation.
- An 'Apply Strategy' button re-runs the simulation with the AI-suggested parameters and updates the chart and table in real time.

Exchange 9: Implementation-Level Specification Additions

PROMPT

More implementation-level detail is needed before this document can be used to prompt Claude. Specifically, the financial model assumptions, grid search parameters, Pareto algorithm steps, edge case handling, and API endpoints need to be precisely specified.

RESPONSE — ChatGPT

The following sections should be added to the requirements document to make it implementation-ready:

Section 2.1 — Financial Model Assumptions

To ensure deterministic computation, the following simplified financial model is used:

- Acquirer Net Income = EPS_a x Shares_a
- Target Net Income = EPS_t x Shares_t
- Shares_a = 100 (normalized baseline)
- Shares_t = Deal Value / Target EPS
- If stock_pct > 0: New Shares Issued = (stock_pct x Deal Value) / Acquirer EPS
- New Income = Income_a + Income_t + Synergy
- New Shares = Shares_a + New Shares Issued
- EPS_post = New Income / New Shares

Section 3.1 — Grid Search Resolution

The decision space is discretized as follows:

- Cash % (C): 0 to 100 in steps of 10
- Synergy (Y): 3 levels — Low (0), Base (user input), High (1.5x Base)

Total scenarios: approximately 30. This ensures computational efficiency while maintaining adequate coverage of the solution space.

Section 3.2 — Pareto Frontier Extraction Algorithm

A scenario i is considered dominated if there exists another scenario j such that EPS_j >= EPS_i AND Sensitivity_j <= Sensitivity_i, with at least one strict inequality.

Algorithm:

26. Initialize empty Pareto set.
27. For each scenario, compare it against all others. If it is dominated, discard it.
28. The remaining scenarios form the Pareto frontier.
29. Sort the Pareto set by EPS descending.
30. Return the top 3 to 5 scenarios.

Section 3.3 — Edge Case Handling

- If EPS_a = 0, return an error (invalid base company).
- If Deal Value = 0, skip the simulation.
- If stock_pct = 0, no new shares are issued.
- If synergy < 0, allow the scenario but mark it as a negative synergy case.
- If delta Y = 0 in the sensitivity calculation, set sensitivity = 0 to prevent division by zero.

Section 5.1 — Backend API Endpoints

```
POST /simulate
Input:  { eps_a: float, eps_t: float, deal_value: float, synergy: float }
Output: { pareto_scenarios: [...], all_scenarios_count: int }
```

```
POST /interpret
Input:  { pareto_scenarios: [...] }
Output: { strategies: [...] }
```

PART 2: Implementation Phase — Claude

The following exchanges document the implementation phase of the project, conducted using Claude (claude.ai). The project requirements document from Part 1 was attached as context for both exchanges.

Exchange 1: Initial Full-Stack Implementation

PROMPT

I want to build a full-stack website based on the attached project requirements document. Please generate the implementation.

RESPONSE — Claude

The following is the complete M&A Quant Optimizer built as a React application. It implements the full optimization engine, Pareto frontier visualization, and Claude API integration as a single component.

What is implemented:

- Optimization Engine — deterministic financial model running a grid search across all Cash % and Synergy combinations (approximately 30 scenarios), enforcing three hard constraints (financing completeness, dilution floor at 90%, synergy realism tiers), computing normalized elasticity sensitivity scores, and extracting the Pareto frontier using the dominance algorithm.
- Pareto Visualizer — a Recharts scatter plot rendering all scenarios as faint dots, with the top 3 Pareto-optimal points highlighted in blue, purple, and green with labeled tooltips.
- AI Strategy Cards — after simulation, the app calls the Claude API with the exact JSON payload schema and system prompt. Each card shows Cash/Stock mix, EPS, sensitivity, synergy, and a 2-sentence AI-generated strategic insight.

Usage: enter Acquirer EPS, Target EPS, Deal Value, and adjust the synergy slider, then click Simulate. The AI cards populate automatically after the chart renders.

```
import { useState, useCallback } from "react";

import { ScatterChart, Scatter, XAxis, YAxis, CartesianGrid, Tooltip,
ResponsiveContainer, ReferenceLine } from "recharts";

// ─── Financial Engine ──────────────────────────────────────────────

function computeEPS(epsA, epsT, dealValue, cashPct, synergy) {

  const sharesA = 100;

  const incomeA = epsA * sharesA;

  const incomeT = epsT * (dealValue / epsT); // = dealValue (normalized)

  const stockPct = 100 - cashPct;
```

```
  const newSharesIssued = stockPct > 0 ? (stockPct / 100 * dealValue) / epsA : 0;

  const newIncome = incomeA + incomeT + synergy;

  const newShares = sharesA + newSharesIssued;

  if (newShares === 0) return null;

  return newIncome / newShares;

}

function computeSensitivity(epsA, epsT, dealValue, cashPct, baseEPS, baseSynergy) {

  if (baseSynergy === 0) return 0;

  const delta = baseSynergy * 0.1;

  const epsHigh = computeEPS(epsA, epsT, dealValue, cashPct, baseSynergy + delta);

  const epsLow = computeEPS(epsA, epsT, dealValue, cashPct, baseSynergy - delta);

  if (!epsHigh || !epsLow || baseEPS === 0) return 0;

  const deltaEPS = epsHigh - epsLow;

  const deltaY = 2 * delta;

  return Math.abs((deltaEPS / baseEPS) / (deltaY / baseSynergy));

}

function runOptimization(epsA, epsT, dealValue, baseSynergy) {

  if (!epsA || !epsT || !dealValue) return { all: [], pareto: [] };

  const synergyLevels = [0, baseSynergy, baseSynergy * 1.5];

  const all = [];

  for (let cashPct = 0; cashPct <= 100; cashPct += 10) {

    for (const syn of synergyLevels) {

      const epsPost = computeEPS(epsA, epsT, dealValue, cashPct, syn);

      if (!epsPost) continue;

      if (epsPost < epsA * 0.90) continue; // dilution floor

      const sens = computeSensitivity(epsA, epsT, dealValue, cashPct, epsA,
baseSynergy);

      all.push({ cashPct, stockPct: 100 - cashPct, synergy: syn, eps: epsPost,
sensitivity: +sens.toFixed(3) });

    }

  }

  // Pareto frontier

  const pareto = all.filter((a) => !all.some((b) => b.eps >= a.eps && b.sensitivity <=
a.sensitivity && (b.eps > a.eps || b.sensitivity < a.sensitivity)));

  pareto.sort((a, b) => b.eps - a.eps);

  const top = pareto.slice(0, 5);
```

```
  // Label top 3

  const labels = ["Maximum Return", "Balanced Frontier", "Maximum Stability"];

  top.forEach((s, i) => { s.label = labels[i] || `Scenario ${i + 1}`; });

  return { all, pareto: top };

}

// ── AI Strategy Cards ──────────────────────────────────────────────

async function fetchStrategies(baseEPS, paretoScenarios) {

  const payload = { base_eps: baseEPS, pareto_scenarios: paretoScenarios.map(s => ({
type: s.label, cash_pct: s.cashPct, stock_pct: s.stockPct, eps: +s.eps.toFixed(2),
sensitivity: s.sensitivity })) };

  const systemPrompt = `You are an elite quantitative M&A analyst. You will receive a
JSON payload with Pareto-optimal deal structures. Return ONLY a valid JSON array (no
markdown, no preamble) with one object per scenario: { "type": string, "insight": string
}. The insight must be exactly 2 sentences explaining the EPS vs sensitivity trade-off
using the provided numbers.`;

  const res = await fetch("https://api.anthropic.com/v1/messages", {

    method: "POST",

    headers: { "Content-Type": "application/json" },

    body: JSON.stringify({ model: "claude-sonnet-4-20250514", max_tokens: 1000, system:
systemPrompt, messages: [{ role: "user", content: JSON.stringify(payload) }] }),

  });

  const data = await res.json();

  const text = data.content?.find(b => b.type === "text")?.text || "[]";

  const clean = text.replace(/```json|```/g, "").trim();

  return JSON.parse(clean);

}

// ── Components ─────────────────────────────────────────────────────

const CARD COLORS = ["#378ADD", "#7F77DD", "#1D9E75"];

const CARD_ICONS = ["↑", "⊙", "◈"];

function StrategyCard({ scenario, insight, color, icon, loading }) {

  return (

    <div style={{ background: "var(--color-background-primary)", border: `1.5px solid
${color}33`, borderRadius: "var(--border-radius-lg)", padding: "1.25rem", display:
"flex", flexDirection: "column", gap: 8, flex: 1, minWidth: 220 }}>

      <div style={{ display: "flex", alignItems: "center", gap: 8 }}>

        <span style={{ width: 28, height: 28, borderRadius: "50%", background: color +
"22", color, display: "flex", alignItems: "center", justifyContent: "center", fontSize:
14, fontWeight: 500 }}>{icon}</span>

        <span style={{ fontWeight: 500, fontSize: 13, color: "var(--color-text-primary)"
}}>{scenario.label}</span>

      </div>
```

```
      <div style={{ display: "grid", gridTemplateColumns: "1fr 1fr", gap: 8 }}>

        {[["Cash / Stock", `${scenario.cashPct}% / ${scenario.stockPct}%`], ["Post-Deal
EPS", `$${scenario.eps.toFixed(2)}`], ["Sensitivity", scenario.sensitivity.toFixed(2)],
["Synergy", `$${scenario.synergy.toFixed(0)}M`]].map(([label, val]) => (

          <div key={label} style={{ background: "var(--color-background-secondary)",
borderRadius: "var(--border-radius-md)", padding: "8px 10px" }}>

            <div style={{ fontSize: 11, color: "var(--color-text-secondary)",
marginBottom: 2 }}>{label}</div>

            <div style={{ fontWeight: 500, fontSize: 14, color: "var(--color-text-
primary)" }}>{val}</div>

          </div>

        ))}

      </div>

      <div style={{ fontSize: 12, color: "var(--color-text-secondary)", lineHeight: 1.6,
borderTop: "0.5px solid var(--color-border-tertiary)", paddingTop: 8, minHeight: 48 }}>

        {loading ? <span style={{ color: color, opacity: 0.7 }}>Generating AI
insight…</span> : (insight || "—")}

      </div>

    </div>

  );

}

const CustomDot = (props) => {

  const { cx, cy, payload } = props;

  if (payload.isPareto) {

    const color = CARD_COLORS[payload.paretoIdx] || "#888";

    return <circle cx={cx} cy={cy} r={8} fill={color} stroke="#fff" strokeWidth={1.5}
/>;

  }

  return <circle cx={cx} cy={cy} r={3} fill="var(--color-text-secondary)" opacity={0.25}
/>;

};

const CustomTooltip = ({ active, payload }) => {

  if (!active || !payload?.length) return null;

  const d = payload[0].payload;

  return (

    <div style={{ background: "var(--color-background-primary)", border: "0.5px solid
var(--color-border-secondary)", borderRadius: "var(--border-radius-md)", padding: "8px
12px", fontSize: 12 }}>

      <div style={{ fontWeight: 500, marginBottom: 4, color: "var(--color-text-primary)"
}}>{d.label || `${d.cashPct}% Cash`}</div>

      <div style={{ color: "var(--color-text-secondary)" }}>EPS: <b style={{ color:
"var(--color-text-primary)" }}>${d.eps.toFixed(3)}</b></div>
```

```
      <div style={{ color: "var(--color-text-secondary)" }}>Sensitivity: <b style={{
color: "var(--color-text-primary)" }}>{d.sensitivity.toFixed(3)}</b></div>

      <div style={{ color: "var(--color-text-secondary)" }}>Synergy: <b style={{ color:
"var(--color-text-primary)" }}>${d.synergy.toFixed(0)}M</b></div>

    </div>

  );

};

// ─── Main App ──────────────────────────────────────────────────────────

export default function App() {

  const [epsA, setEpsA] = useState(5.0);

  const [epsT, setEpsT] = useState(3.0);

  const [dealValue, setDealValue] = useState(500);

  const [synergy, setSynergy] = useState(50);

  const [result, setResult] = useState(null);

  const [strategies, setStrategies] = useState([]);

  const [aiLoading, setAiLoading] = useState(false);

  const [error, setError] = useState("");

  const [ran, setRan] = useState(false);

  const simulate = useCallback(async () => {

    setError("");

    if (!epsA || !epsT || !dealValue) { setError("All input fields are required.");
return; }

    const { all, pareto } = runOptimization(epsA, epsT, dealValue, synergy);

    if (pareto.length === 0) { setError("No feasible scenarios found. Try adjusting your
inputs."); setResult(null); return; }

    setResult({ all, pareto });

    setRan(true);

    setStrategies([]);

    setAiLoading(true);

    try {

      const insights = await fetchStrategies(epsA, pareto);

      setStrategies(insights);

    } catch (e) {

      setStrategies([]);

    }

    setAiLoading(false);
```

```
  }, [epsA, epsT, dealValue, synergy]);

  // Merge pareto flags into chart data

  const chartData = result ? [

    ...result.all.map(s => ({ ...s, isPareto: false })),

    ...result.pareto.map((s, i) => ({ ...s, isPareto: true, paretoIdx: i })),

  ] : [];

  const inp = (label, val, setter, min, max, step, prefix = "$") => (

    <div style={{ marginBottom: 16 }}>

      <label style={{ fontSize: 12, color: "var(--color-text-secondary)", display:
"block", marginBottom: 4 }}>{label}</label>

      <div style={{ display: "flex", alignItems: "center", gap: 6, background: "var(--
color-background-secondary)", border: "0.5px solid var(--color-border-tertiary)",
borderRadius: "var(--border-radius-md)", padding: "6px 10px" }}>

        <span style={{ fontSize: 13, color: "var(--color-text-secondary)"
}}>{prefix}</span>

        <input type="number" value={val} min={min} max={max} step={step} onChange={e =>
setter(+e.target.value)} style={{ border: "none", background: "transparent", fontWeight:
500, fontSize: 15, color: "var(--color-text-primary)", width: "100%", outline: "none" }}
/>

      </div>

    </div>

  );

  return (

    <div style={{ fontFamily: "var(--font-sans)", padding: "1.5rem 1rem", maxWidth: 960,
margin: "0 auto" }}>

      <h2 style={{ sr: "only", fontSize: 22, fontWeight: 500, margin: "0 0 0.25rem",
color: "var(--color-text-primary)" }}>M&A Quant Optimizer</h2>

      <p style={{ fontSize: 13, color: "var(--color-text-secondary)", margin: "0 0
1.5rem" }}>Pareto-efficient deal structures · EPS maximization · Sensitivity-
constrained</p>

      <div style={{ display: "flex", gap: "1.5rem", flexWrap: "wrap", alignItems: "flex-
start" }}>

        {/* ── Input Panel ── */}

        <div style={{ minWidth: 220, flex: "0 0 220px", background: "var(--color-
background-primary)", border: "0.5px solid var(--color-border-tertiary)", borderRadius:
"var(--border-radius-lg)", padding: "1.25rem" }}>

          <div style={{ fontSize: 13, fontWeight: 500, color: "var(--color-text-
secondary)", marginBottom: 14, textTransform: "uppercase", letterSpacing: "0.08em",
fontSize: 11 }}>Deal Parameters</div>

          {inp("Acquirer EPS (Base)", epsA, setEpsA, 0.1, 100, 0.1)}

          {inp("Target EPS", epsT, setEpsT, 0.1, 100, 0.1)}

          {inp("Deal Value ($M)", dealValue, setDealValue, 1, 100000, 1, "$")}

          <div style={{ marginBottom: 20 }}>
```

```
            <div style={{ display: "flex", justifyContent: "space-between",
marginBottom: 4 }}>

              <label style={{ fontSize: 12, color: "var(--color-text-secondary)" }}>Base
Synergy ($M)</label>

              <span style={{ fontSize: 12, fontWeight: 500, color: "var(--color-text-
primary)" }}>${synergy}M</span>

            </div>

            <input type="range" min={0} max={500} step={5} value={synergy} onChange={e
=> setSynergy(+e.target.value)} style={{ width: "100%" }} />

            <div style={{ display: "flex", justifyContent: "space-between", fontSize:
10, color: "var(--color-text-secondary)", marginTop: 2 }}>

              <span>$0</span><span>$500M</span>

            </div>

          </div>

          <button onClick={simulate} style={{ width: "100%", padding: "10px 0",
borderRadius: "var(--border-radius-md)", background: "#1D9E75", border: "none", color:
"#fff", fontWeight: 500, fontSize: 14, cursor: "pointer", letterSpacing: "0.02em" }}>

            Simulate Optimal Deals ↗

          </button>

          {error && <div style={{ marginTop: 10, fontSize: 12, color: "var(--color-text-
danger)", lineHeight: 1.5 }}>{error}</div>}

          {result && (

            <div style={{ marginTop: 16, fontSize: 12, color: "var(--color-text-
secondary)", lineHeight: 1.8 }}>

              <div>Total scenarios: <b style={{ color: "var(--color-text-primary)"
}}>{result.all.length}</b></div>

              <div>Pareto-efficient: <b style={{ color: "#1D9E75"
}}>{result.pareto.length}</b></div>

            </div>

          )}

        </div>

        {/* ── Chart Panel ── */}

        <div style={{ flex: 1, minWidth: 300, background: "var(--color-background-
primary)", border: "0.5px solid var(--color-border-tertiary)", borderRadius: "var(--
border-radius-lg)", padding: "1.25rem" }}>

          <div style={{ fontSize: 11, fontWeight: 500, color: "var(--color-text-
secondary)", textTransform: "uppercase", letterSpacing: "0.08em", marginBottom: 12
}}>Pareto Frontier — Risk vs. Return</div>

          {!ran ? (

            <div style={{ height: 300, display: "flex", alignItems: "center",
justifyContent: "center", color: "var(--color-text-secondary)", fontSize: 13 }}>Enter
parameters and simulate to see the Pareto frontier</div>

          ) : (

            <>
```

```
            <div style={{ display: "flex", gap: 16, marginBottom: 12, flexWrap: "wrap"
}}>

              {result.pareto.map((s, i) => (

                <span key={i} style={{ display: "flex", alignItems: "center", gap: 5,
fontSize: 11, color: "var(--color-text-secondary)" }}>

                  <span style={{ width: 10, height: 10, borderRadius: "50%",
background: CARD_COLORS[i] || "#888", display: "inline-block" }}></span>

                  {s.label}

                </span>

              ))}

              <span style={{ display: "flex", alignItems: "center", gap: 5, fontSize:
11, color: "var(--color-text-secondary)" }}>

                <span style={{ width: 10, height: 10, borderRadius: "50%", background:
"var(--color-text-secondary)", opacity: 0.4, display: "inline-block" }}></span>

                All scenarios

              </span>

            </div>

            <div style={{ height: 300 }}>

              <ResponsiveContainer width="100%" height="100%">

                <ScatterChart margin={{ top: 10, right: 20, bottom: 20, left: 0 }}>

                  <CartesianGrid strokeDasharray="3 3" stroke="var(--color-border-
tertiary)" />

                  <XAxis dataKey="sensitivity" name="Sensitivity (Risk)" type="number"
domain={['auto', 'auto']} label={{ value: "Sensitivity (Risk →)", position:
"insideBottom", offset: -10, fontSize: 11, fill: "var(--color-text-secondary)" }}
tick={{ fontSize: 11, fill: "var(--color-text-secondary)" }} />

                  <YAxis dataKey="eps" name="Post-Deal EPS" type="number"
domain={['auto', 'auto']} label={{ value: "EPS (Return ↑)", angle: -90, position:
"insideLeft", offset: 15, fontSize: 11, fill: "var(--color-text-secondary)" }} tick={{
fontSize: 11, fill: "var(--color-text-secondary)" }} />

                  <Tooltip content={<CustomTooltip />} />

                  <Scatter data={chartData} shape={<CustomDot />} />

                </ScatterChart>

              </ResponsiveContainer>

            </div>

          </>

        )}

      </div>

    </div>

    {/* ── Strategy Cards ── */}

    {ran && result?.pareto?.length > 0 && (

      <div style={{ marginTop: "1.5rem" }}>
```

```
          <div style={{ fontSize: 11, fontWeight: 500, color: "var(--color-text-
secondary)", textTransform: "uppercase", letterSpacing: "0.08em", marginBottom: 12 }}>AI
Strategic Analysis · Top Pareto Configurations</div>

          <div style={{ display: "flex", gap: 16, flexWrap: "wrap" }}>

            {result.pareto.slice(0, 3).map((s, i) => {

              const insight = strategies.find(st => st.type === s.label)?.insight ||
null;

              return <StrategyCard key={i} scenario={s} insight={insight}
color={CARD COLORS[i]} icon={CARD ICONS[i]} loading={aiLoading && !insight} />;

            })}

          </div>

        </div>

      )}

      {/* ── Constraints Legend ── */}

      <div style={{ marginTop: "1.5rem", display: "flex", gap: 20, flexWrap: "wrap" }}>

        {[["Financing", "C + S = 100%"], ["Dilution Floor", "EPS_post ≥ 0.90 ×
EPS_base"], ["Synergy Realism", "3 levels: Low · Base · 1.5×"]].map(([k, v]) => (

          <div key={k} style={{ fontSize: 11, color: "var(--color-text-secondary)" }}>

            <span style={{ fontWeight: 500, color: "var(--color-text-primary)" }}>{k}:
</span>{v}

          </div>

        ))}

      </div>

    </div>

  );

}
```

Exchange 2: Frontend Enhancement Request

PROMPT

The core logic of the M&A Quant Optimizer is working correctly. The following upgrade is required for the frontend, without changing any of the underlying optimization logic.

Design System:

- Use a clean, premium fintech-style UI — not a basic student-project aesthetic.
- Light theme with subtle depth. Soft gradients, minimal glassmorphism, and smooth shadows.
- Rounded corners, consistent spacing, and clear typographic hierarchy across titles, section labels, and data labels.

Layout Structure:

- Left: compact input control panel (sticky).
- Centre: main visualization (Pareto scatter plot).
- Bottom or right: AI strategy cards for the top 3 Pareto scenarios.
- Summary bar showing: Total Scenarios, Pareto Optimal Count, Best EPS, and Minimum Sensitivity.

Interactions:

- Smooth transition when the user clicks Simulate, with a loading animation simulating computation for approximately 1 to 1.5 seconds.
- Chart should animate on render: points fade in, Pareto points highlight distinctly.
- Tooltip on hover showing EPS, sensitivity, cash %, and synergy.
- Strategy cards: hover elevation with subtle glow on the active card.

Visual Enhancements:

- Normal scenarios rendered as light grey dots; Pareto points in 3 distinct colours (blue, purple, green).
- Legend clearly identifying all data series.
- Axis labels: 'Sensitivity (Risk)' and 'EPS (Return)'.
- Best scenario visually highlighted.

AI Section:

- Each strategy card displays: scenario label (Maximum Return / Balanced Frontier / Maximum Stability), Cash vs. Stock %, EPS, Sensitivity, and the 2-sentence AI insight.
- Loading state displayed while AI insights are being generated.

Code Requirements:

- All logic in a single React component file or clean modular components.
- Functional components with hooks only. Do not modify the existing optimization logic.
- Inline styles or a minimal styling system — no heavy CSS frameworks.
- Full working code only — no partial snippets or placeholders.

RESPONSE — Claude

The following is the complete upgraded M&A Quant Optimizer. The financial engine, Pareto extraction logic, and Claude API integration are unchanged. All frontend elements have been rebuilt with a premium fintech-style design system including a sticky sidebar, animated scatter plot, stat summary bar, and styled strategy cards.

```
import { useState, useCallback, useEffect, useRef } from "react";

import {

  ScatterChart, Scatter, XAxis, YAxis, CartesianGrid,
```

```
  Tooltip, ResponsiveContainer, Cell

} from "recharts";

/* ─────────────────────────────────────────────

   GLOBAL STYLES (injected once)

──────────────────────────────────────────────── */

const STYLES = `

  @import
url('https://fonts.googleapis.com/css2?family=DM+Sans:ital,opsz,wght@0,9..40,300;0,9..40,400;0,9..40,5
00;0,9..40,600;1,9..40,300&family=DM+Mono:wght@400;500&display=swap');

  *, *::before, *::after { box-sizing: border-box; margin: 0; padding: 0; }

  :root {

    --bg:          #F4F6FA;

    --surface:     #FFFFFF;

    --surface-2:   #F0F3F9;

    --border:      rgba(15,23,42,0.08);

    --border-mid:  rgba(15,23,42,0.14);

    --text-1:      #0F172A;

    --text-2:      #475569;

    --text-3:      #94A3B8;

    --accent:      #0EA5E9;

    --accent-bg:   rgba(14,165,233,0.08);

    --emerald:     #059669;

    --emerald-bg:  rgba(5,150,105,0.08);

    --violet:      #7C3AED;

    --violet-bg:   rgba(124,58,237,0.08);

    --amber:       #D97706;

    --amber-bg:    rgba(217,119,6,0.08);

    --danger:      #DC2626;

    --danger-bg:   rgba(220,38,38,0.08);

    --shadow-sm:   0 1px 3px rgba(15,23,42,0.06), 0 1px 2px rgba(15,23,42,0.04);

    --shadow-md:   0 4px 16px rgba(15,23,42,0.08), 0 1px 4px rgba(15,23,42,0.04);

    --shadow-lg:   0 12px 40px rgba(15,23,42,0.12), 0 4px 12px rgba(15,23,42,0.06);

    --radius-sm:   8px;

    --radius-md:   12px;

    --radius-lg:   18px;
```

```
  --radius-xl:   24px;
  --font:        'DM Sans', system-ui, sans-serif;
  --mono:        'DM Mono', monospace;
}
body { font-family: var(--font); background: var(--bg); color: var(--text-1); }
.maq-root {
  min-height: 100vh;
  display: grid;
  grid-template-rows: auto 1fr auto;
  grid-template-columns: 300px 1fr;
  grid-template-areas:
    "header header"
    "sidebar main"
    "sidebar footer";
  gap: 0;
}
/* ── Header ── */
.maq-header {
  grid-area: header;
  background: var(--surface);
  border-bottom: 1px solid var(--border);
  padding: 0 32px;
  height: 64px;
  display: flex;
  align-items: center;
  justify-content: space-between;
  position: sticky;
  top: 0;
  z-index: 100;
}
.maq-logo {
  display: flex; align-items: center; gap: 10px;
}
.maq-logo-mark {
  width: 32px; height: 32px; border-radius: 9px;
```

```
  background: linear-gradient(135deg, #0EA5E9 0%, #7C3AED 100%);
  display: flex; align-items: center; justify-content: center;
}
.maq-logo-mark svg { width: 16px; height: 16px; }
.maq-logo-title {
  font-size: 15px; font-weight: 600; letter-spacing: -0.02em; color: var(--text-1);
}
.maq-logo-sub { font-size: 12px; color: var(--text-3); margin-left: 2px; }
.maq-header-meta {
  display: flex; gap: 24px; align-items: center;
}
.maq-badge {
  display: inline-flex; align-items: center; gap: 5px;
  font-size: 11px; font-weight: 500; color: var(--text-2);
  background: var(--surface-2); border: 1px solid var(--border);
  border-radius: 20px; padding: 4px 10px;
}
.maq-badge-dot {
  width: 6px; height: 6px; border-radius: 50%;
  background: var(--emerald);
  animation: pulse-dot 2s ease-in-out infinite;
}
@keyframes pulse-dot {
  0%, 100% { opacity: 1; }
  50% { opacity: 0.4; }
}
/* ── Sidebar ── */
.maq-sidebar {
  grid-area: sidebar;
  background: var(--surface);
  border-right: 1px solid var(--border);
  padding: 28px 20px;
  position: sticky;
  top: 64px;
```

```
  height: calc(100vh - 64px);
  overflow-y: auto;
  display: flex;
  flex-direction: column;
  gap: 24px;
}
.maq-section-label {
  font-size: 10px; font-weight: 600; letter-spacing: 0.1em;
  text-transform: uppercase; color: var(--text-3);
  margin-bottom: 14px;
}
/* Input field */
.maq-field { margin-bottom: 16px; }
.maq-field label {
  font-size: 12px; font-weight: 500; color: var(--text-2);
  display: flex; justify-content: space-between; margin-bottom: 6px;
}
.maq-field label span { font-family: var(--mono); font-size: 11px; color: var(--text-3); }
.maq-input-wrap {
  position: relative;
  background: var(--surface-2);
  border: 1px solid var(--border);
  border-radius: var(--radius-sm);
  display: flex; align-items: center;
  transition: border-color 0.15s, box-shadow 0.15s;
}
.maq-input-wrap:focus-within {
  border-color: var(--accent);
  box-shadow: 0 0 0 3px rgba(14,165,233,0.12);
}
.maq-input-prefix {
  padding: 0 10px; font-size: 13px; color: var(--text-3); font-family: var(--mono);
  pointer-events: none; user-select: none;
}
.maq-input-wrap input {
```

```
  flex: 1; background: transparent; border: none; outline: none;
  padding: 9px 10px 9px 0; font-size: 14px; font-weight: 500;
  font-family: var(--mono); color: var(--text-1);
}
.maq-input-wrap input::-webkit-inner-spin-button { display: none; }
/* Slider */
.maq-slider-wrap { margin-bottom: 20px; }
.maq-slider-header {
  display: flex; justify-content: space-between; align-items: baseline;
  margin-bottom: 10px;
}
.maq-slider-label { font-size: 12px; font-weight: 500; color: var(--text-2); }
.maq-slider-val {
  font-family: var(--mono); font-size: 13px; font-weight: 500;
  color: var(--text-1);
  background: var(--surface-2); border: 1px solid var(--border);
  border-radius: 6px; padding: 2px 8px;
}
.maq-slider { width: 100%; accent-color: var(--accent); cursor: pointer; }
.maq-slider-ticks {
  display: flex; justify-content: space-between;
  font-size: 10px; color: var(--text-3); margin-top: 4px;
  font-family: var(--mono);
}
/* CTA Button */
.maq-btn {
  width: 100%; padding: 12px;
  border-radius: var(--radius-md);
  border: none; cursor: pointer;
  font-family: var(--font); font-size: 14px; font-weight: 600;
  letter-spacing: -0.01em;
  transition: all 0.2s;
  position: relative; overflow: hidden;
}
```

```
.maq-btn-primary {
  background: linear-gradient(135deg, #0EA5E9 0%, #0284C7 100%);
  color: #fff;
  box-shadow: 0 4px 12px rgba(14,165,233,0.3), 0 1px 3px rgba(14,165,233,0.2);
}
.maq-btn-primary:hover:not(:disabled) {
  transform: translateY(-1px);
  box-shadow: 0 8px 20px rgba(14,165,233,0.35), 0 2px 6px rgba(14,165,233,0.2);
}
.maq-btn-primary:active:not(:disabled) { transform: translateY(0); }
.maq-btn-primary:disabled {
  background: linear-gradient(135deg, #94A3B8 0%, #CBD5E1 100%);
  box-shadow: none; cursor: not-allowed;
}
.maq-btn-shimmer::after {
  content: ''; position: absolute; inset: 0;
  background: linear-gradient(90deg, transparent 0%, rgba(255,255,255,0.25) 50%, transparent 100%);
  transform: translateX(-100%);
  animation: shimmer-slide 1.5s ease-in-out infinite;
}
@keyframes shimmer-slide {
  to { transform: translateX(100%); }
}
/* Constraint pills */
.maq-constraints {
  display: flex; flex-direction: column; gap: 8px; margin-top: 4px;
}
.maq-constraint {
  display: flex; align-items: flex-start; gap: 8px;
  font-size: 11px; color: var(--text-2); line-height: 1.4;
}
.maq-constraint-icon {
  width: 16px; height: 16px; border-radius: 4px;
  background: var(--emerald-bg); flex-shrink: 0; margin-top: 1px;
  display: flex; align-items: center; justify-content: center;
```

```
}
.maq-constraint-icon svg { width: 9px; height: 9px; }
.maq-constraint strong { color: var(--text-1); font-weight: 500; }
/* ── Main area ── */
.maq-main {
  grid-area: main;
  padding: 28px 28px 0;
  display: flex; flex-direction: column; gap: 20px;
  min-height: 0;
}
/* Stats bar */
.maq-stats {
  display: grid; grid-template-columns: repeat(4, 1fr); gap: 12px;
}
.maq-stat-card {
  background: var(--surface);
  border: 1px solid var(--border);
  border-radius: var(--radius-md);
  padding: 16px 18px;
  box-shadow: var(--shadow-sm);
  transition: box-shadow 0.2s, transform 0.2s;
}
.maq-stat-card:hover { box-shadow: var(--shadow-md); transform: translateY(-1px); }
.maq-stat-label {
  font-size: 11px; font-weight: 500; color: var(--text-3);
  text-transform: uppercase; letter-spacing: 0.06em; margin-bottom: 6px;
}
.maq-stat-val {
  font-family: var(--mono); font-size: 24px; font-weight: 500;
  color: var(--text-1); letter-spacing: -0.02em;
  transition: color 0.3s;
}
.maq-stat-val.colored { color: var(--emerald); }
.maq-stat-sub { font-size: 11px; color: var(--text-3); margin-top: 3px; }
```

```
/* Chart card */
.maq-chart-card {
  background: var(--surface);
  border: 1px solid var(--border);
  border-radius: var(--radius-lg);
  box-shadow: var(--shadow-md);
  padding: 24px;
  flex: 1; min-height: 360px;
}
.maq-chart-header {
  display: flex; justify-content: space-between; align-items: flex-start;
  margin-bottom: 20px;
}
.maq-chart-title { font-size: 15px; font-weight: 600; color: var(--text-1); }
.maq-chart-subtitle { font-size: 12px; color: var(--text-3); margin-top: 2px; }
.maq-legend {
  display: flex; gap: 16px; align-items: center; flex-wrap: wrap;
}
.maq-legend-item {
  display: flex; align-items: center; gap: 5px;
  font-size: 11px; color: var(--text-2);
}
.maq-legend-dot {
  width: 8px; height: 8px; border-radius: 50%;
}
/* Empty state */
.maq-empty {
  height: 100%; display: flex; flex-direction: column;
  align-items: center; justify-content: center; gap: 12px;
  padding: 60px 20px;
}
.maq-empty-icon {
  width: 56px; height: 56px; border-radius: 16px;
  background: var(--surface-2); border: 1px solid var(--border);
  display: flex; align-items: center; justify-content: center;
```

```
}
.maq-empty-title { font-size: 15px; font-weight: 500; color: var(--text-2); }
.maq-empty-sub { font-size: 13px; color: var(--text-3); text-align: center; max-width: 280px; }
/* Loading skeleton */
.maq-skeleton {
  border-radius: 6px; background: var(--surface-2);
  animation: skeleton-pulse 1.5s ease-in-out infinite;
}
@keyframes skeleton-pulse {
  0%, 100% { opacity: 1; }
  50% { opacity: 0.4; }
}
/* ── Strategy Cards ── */
.maq-cards-section {
  padding: 0 28px 28px;
}
.maq-cards-header {
  display: flex; align-items: center; justify-content: space-between;
  margin-bottom: 14px;
}
.maq-cards-title { font-size: 13px; font-weight: 600; color: var(--text-1); }
.maq-ai-badge {
  display: inline-flex; align-items: center; gap: 5px;
  font-size: 10px; font-weight: 600; letter-spacing: 0.08em;
  text-transform: uppercase; color: var(--violet);
  background: var(--violet-bg); border-radius: 20px; padding: 3px 10px;
}
.maq-cards-grid {
  display: grid; grid-template-columns: repeat(3, 1fr); gap: 16px;
}
.maq-scenario-card {
  background: var(--surface);
  border: 1px solid var(--border);
  border-radius: var(--radius-lg);
```

```
    padding: 20px;
    box-shadow: var(--shadow-sm);
    transition: box-shadow 0.25s, transform 0.25s, border-color 0.25s;
    cursor: default;
    position: relative; overflow: hidden;
  }
  .maq-scenario-card::before {
    content: ''; position: absolute; top: 0; left: 0; right: 0; height: 3px;
    background: var(--card-accent, #94A3B8);
    border-radius: var(--radius-lg) var(--radius-lg) 0 0;
  }
  .maq-scenario-card:hover {
    box-shadow: var(--shadow-lg);
    transform: translateY(-3px);
    border-color: var(--card-accent-border, var(--border-mid));
  }
  .maq-card-header { display: flex; align-items: center; gap: 10px; margin-bottom: 16px; }
  .maq-card-icon {
    width: 34px; height: 34px; border-radius: 10px;
    background: var(--card-icon-bg, var(--surface-2));
    display: flex; align-items: center; justify-content: center;
    font-size: 15px; flex-shrink: 0;
  }
  .maq-card-label { font-size: 14px; font-weight: 600; color: var(--text-1); }
  .maq-card-type { font-size: 11px; color: var(--text-3); margin-top: 1px; }
  .maq-card-metrics {
    display: grid; grid-template-columns: 1fr 1fr; gap: 8px; margin-bottom: 14px;
  }
  .maq-metric {
    background: var(--surface-2); border-radius: var(--radius-sm);
    padding: 9px 11px;
  }
  .maq-metric-label { font-size: 10px; font-weight: 500; color: var(--text-3); margin-bottom: 3px;
text-transform: uppercase; letter-spacing: 0.05em; }
  .maq-metric-val { font-family: var(--mono); font-size: 15px; font-weight: 500; color: var(--text-1);
}
```

```
.maq-metric-val.accent { color: var(--card-color, var(--text-1)); }
.maq-card-divider {
  height: 1px; background: var(--border); margin-bottom: 12px;
}
.maq-card-insight {
  font-size: 12px; line-height: 1.65; color: var(--text-2);
  min-height: 48px;
}
.maq-insight-loading {
  display: flex; align-items: center; gap: 6px;
  font-size: 11px; color: var(--text-3); font-style: italic;
}
.maq-insight-dots span {
  display: inline-block; animation: dot-bounce 1.2s ease-in-out infinite;
}
.maq-insight-dots span:nth-child(2) { animation-delay: 0.2s; }
.maq-insight-dots span:nth-child(3) { animation-delay: 0.4s; }
@keyframes dot-bounce {
  0%, 80%, 100% { transform: translateY(0); }
  40% { transform: translateY(-4px); }
}
/* Error */
.maq-error {
  display: flex; align-items: flex-start; gap: 8px;
  background: var(--danger-bg); border: 1px solid rgba(220,38,38,0.2);
  border-radius: var(--radius-sm); padding: 10px 12px;
  font-size: 12px; color: var(--danger); margin-top: 12px; line-height: 1.5;
}
/* Compute loading overlay for chart */
.maq-compute-overlay {
  position: absolute; inset: 0;
  background: rgba(255,255,255,0.85);
  backdrop-filter: blur(4px);
  display: flex; flex-direction: column;
```

```
  align-items: center; justify-content: center; gap: 14px;
  border-radius: var(--radius-lg);
  z-index: 10;
}
.maq-compute-spinner {
  width: 36px; height: 36px; border-radius: 50%;
  border: 2px solid var(--border);
  border-top-color: var(--accent);
  animation: spin 0.7s linear infinite;
}
@keyframes spin { to { transform: rotate(360deg); } }
.maq-compute-label { font-size: 13px; font-weight: 500; color: var(--text-2); }
.maq-compute-sub { font-size: 11px; color: var(--text-3); }
/* Point fade-in animation */
.recharts-symbols { animation: points-in 0.6s ease-out; }
@keyframes points-in {
  from { opacity: 0; transform: scale(0.6); }
  to { opacity: 1; transform: scale(1); }
}
/* Tooltip */
.maq-tt {
  background: var(--surface); border: 1px solid var(--border-mid);
  border-radius: var(--radius-md); padding: 12px 14px;
  box-shadow: var(--shadow-lg); min-width: 180px;
}
.maq-tt-title { font-size: 12px; font-weight: 600; color: var(--text-1); margin-bottom: 8px; }
.maq-tt-row {
  display: flex; justify-content: space-between; align-items: center;
  gap: 16px; padding: 2px 0; font-size: 11px;
}
.maq-tt-key { color: var(--text-3); }
.maq-tt-val { font-family: var(--mono); font-weight: 500; color: var(--text-1); }
/* Footer */
.maq-footer {
  grid-area: footer;
```

```
    padding: 12px 28px;
    border-top: 1px solid var(--border);
    background: var(--surface);
    font-size: 11px; color: var(--text-3);
    display: flex; align-items: center; justify-content: space-between;
  }
  .maq-footer-pills { display: flex; gap: 12px; }
  .maq-footer-pill {
    background: var(--surface-2); border: 1px solid var(--border);
    border-radius: 20px; padding: 3px 10px;
    font-family: var(--mono); font-size: 10px; color: var(--text-3);
  }
  /* Responsive */
  @media (max-width: 860px) {
    .maq-root {
      grid-template-columns: 1fr;
      grid-template-areas: "header" "sidebar" "main" "footer";
    }
    .maq-sidebar {
      position: static; height: auto; border-right: none; border-bottom: 1px solid var(--border);
    }
    .maq-stats { grid-template-columns: repeat(2, 1fr); }
    .maq-cards-grid { grid-template-columns: 1fr; }
  }
`;
/* ——————————————————————————————————————
   FINANCIAL ENGINE (unchanged logic)
—————————————————————————————————————— */
function computeEPS(epsA, epsT, dealValue, cashPct, synergy) {
  const sharesA = 100;
  const incomeA = epsA * sharesA;
  const incomeT = dealValue; // epsT * (dealValue/epsT) normalized
  const stockPct = 100 - cashPct;
  const newSharesIssued = stockPct > 0 ? (stockPct / 100 * dealValue) / epsA : 0;
```

```
  const newIncome = incomeA + incomeT + synergy;
  const newShares = sharesA + newSharesIssued;
  if (newShares === 0) return null;
  return newIncome / newShares;
}
function computeSensitivity(epsA, epsT, dealValue, cashPct, baseEPS, baseSynergy) {
  if (baseSynergy === 0) return 0;
  const delta = baseSynergy * 0.1;
  const epsHigh = computeEPS(epsA, epsT, dealValue, cashPct, baseSynergy + delta);
  const epsLow  = computeEPS(epsA, epsT, dealValue, cashPct, baseSynergy - delta);
  if (!epsHigh || !epsLow || baseEPS === 0) return 0;
  return Math.abs(((epsHigh - epsLow) / baseEPS) / ((2 * delta) / baseSynergy));
}
function runOptimization(epsA, epsT, dealValue, baseSynergy) {
  if (!epsA || !epsT || !dealValue) return { all: [], pareto: [] };
  const synergyLevels = [0, baseSynergy, baseSynergy * 1.5];
  const all = [];
  for (let cashPct = 0; cashPct <= 100; cashPct += 10) {
    for (const syn of synergyLevels) {
      const epsPost = computeEPS(epsA, epsT, dealValue, cashPct, syn);
      if (!epsPost) continue;
      if (epsPost < epsA * 0.9) continue;
      const sens = computeSensitivity(epsA, epsT, dealValue, cashPct, epsA, baseSynergy);
      all.push({
        cashPct, stockPct: 100 - cashPct,
        synergy: syn, eps: epsPost,
        sensitivity: parseFloat(sens.toFixed(4)),
      });
    }
  }
  const pareto = all.filter(a =>
    !all.some(b => b.eps >= a.eps && b.sensitivity <= a.sensitivity &&
      (b.eps > a.eps || b.sensitivity < a.sensitivity))
  );
  pareto.sort((a, b) => b.eps - a.eps);
```

```
  const top = pareto.slice(0, 5);
  const labels = ["Maximum Return", "Balanced Frontier", "Maximum Stability"];
  top.forEach((s, i) => { s.label = labels[i] || `Scenario ${i + 1}`; });
  return { all, pareto: top };
}
async function fetchStrategies(baseEPS, paretoScenarios) {
  const payload = {
    base_eps: baseEPS,
    pareto_scenarios: paretoScenarios.map(s => ({
      type: s.label, cash_pct: s.cashPct, stock_pct: s.stockPct,
      eps: +s.eps.toFixed(2), sensitivity: +s.sensitivity.toFixed(3),
    })),
  };
  const system = `You are an elite quantitative M&A analyst. Receive Pareto-optimal deal structures in JSON. Return ONLY a valid JSON array with no markdown or preamble. Each element: { "type": string, "insight": string }. insight = exactly 2 sharp sentences explaining the EPS vs sensitivity trade-off using the provided numbers. Be precise and professional.`;
  const res = await fetch("https://api.anthropic.com/v1/messages", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({
      model: "claude-sonnet-4-20250514", max_tokens: 1000,
      system,
      messages: [{ role: "user", content: JSON.stringify(payload) }],
    }),
  });
  const data = await res.json();
  const text = data.content?.find(b => b.type === "text")?.text || "[]";
  return JSON.parse(text.replace(/```json|```/g, "").trim());
}
/* ─────────────────────────────────────────────
   DESIGN TOKENS
──────────────────────────────────────────── */
const PARETO_COLORS  = ["#0EA5E9", "#7C3AED", "#059669"];
const PARETO BG      = ["rgba(14,165,233,0.1)", "rgba(124,58,237,0.1)", "rgba(5,150,105,0.1)"];
const PARETO_BORDER  = ["rgba(14,165,233,0.3)", "rgba(124,58,237,0.3)", "rgba(5,150,105,0.3)"];
```

```
const PARETO ICONS    = ["📈", "⚖️", "🛡️"];
const PARETO_SUBTYPES = ["High Return · High Risk", "Balanced Trade-off", "Low Risk · Stable EPS"];
/* ──────────────────────────────────────────────
   SUB-COMPONENTS
────────────────────────────────────────────── */
function InputField({ label, prefix = "$", value, setter, min, max, step, hint }) {
  return (
    <div className="maq-field">
      <label>
        {label}
        {hint && <span>{hint}</span>}
      </label>
      <div className="maq-input-wrap">
        {prefix && <span className="maq-input-prefix">{prefix}</span>}
        <input
          type="number" value={value} min={min} max={max} step={step}
          onChange={e => setter(parseFloat(e.target.value) || 0)}
        />
      </div>
    </div>
  );
}
function StatCard({ label, value, sub, colored, skeleton }) {
  return (
    <div className="maq-stat-card">
      <div className="maq-stat-label">{label}</div>
      {skeleton
        ? <div className="maq-skeleton" style={{ height: 28, width: "60%", marginBottom: 4 }} />
        : <div className={`maq-stat-val${colored ? " colored" : ""}`}>{value}</div>
      }
      <div className="maq-stat-sub">{sub}</div>
    </div>
  );
}
```

```
function CustomTooltip({ active, payload }) {
  if (!active || !payload?.length) return null;
  const d = payload[0]?.payload;
  if (!d) return null;
  return (
    <div className="maq-tt">
      <div className="maq-tt-title">{d.isPareto ? d.label : `${d.cashPct}% Cash · ${d.stockPct}% Stock`}</div>
      {[
        ["Post-Deal EPS", `$${d.eps.toFixed(3)}`],
        ["Sensitivity", d.sensitivity.toFixed(3)],
        ["Synergy", `$${d.synergy.toFixed(0)}M`],
        ["Cash / Stock", `${d.cashPct}% / ${d.stockPct}%`],
      ].map(([k, v]) => (
        <div className="maq-tt-row" key={k}>
          <span className="maq-tt-key">{k}</span>
          <span className="maq-tt-val">{v}</span>
        </div>
      ))}
    </div>
  );
}
function ScenarioCard({ scenario, insight, aiLoading, idx }) {
  const color  = PARETO_COLORS[idx]  || "#94A3B8";
  const bg     = PARETO_BG[idx]     || "rgba(148,163,184,0.1)";
  const border = PARETO_BORDER[idx] || "rgba(148,163,184,0.3)";
  const icon   = PARETO_ICONS[idx]  || "◈";
  const subtype = PARETO SUBTYPES[idx] || "";
  return (
    <div
      className="maq-scenario-card"
      style={{ "--card-accent": color, "--card-accent-border": border, "--card-icon-bg": bg, "--card-color": color }}
    >
      <div className="maq-card-header">
```

```
        <div className="maq-card-icon">{icon}</div>
        <div>
          <div className="maq-card-label">{scenario.label}</div>
          <div className="maq-card-type">{subtype}</div>
        </div>
      </div>
      <div className="maq-card-metrics">
        <div className="maq-metric">
          <div className="maq-metric-label">Post EPS</div>
          <div className="maq-metric-val accent">${scenario.eps.toFixed(2)}</div>
        </div>
        <div className="maq-metric">
          <div className="maq-metric-label">Sensitivity</div>
          <div className="maq-metric-val">{scenario.sensitivity.toFixed(3)}</div>
        </div>
        <div className="maq-metric">
          <div className="maq-metric-label">Cash Mix</div>
          <div className="maq-metric-val">{scenario.cashPct}%</div>
        </div>
        <div className="maq-metric">
          <div className="maq-metric-label">Synergy</div>
          <div className="maq-metric-val">${scenario.synergy.toFixed(0)}M</div>
        </div>
      </div>
      <div className="maq-card-divider" />
      <div className="maq-card-insight">
        {aiLoading && !insight ? (
          <div className="maq-insight-loading">
            <span style={{ color }}>Generating AI insight</span>
            <span className="maq-insight-dots">
              <span>.</span><span>.</span><span>.</span>
            </span>
          </div>
        ) : (
          insight || <span style={{ color: "var(--text-3)", fontStyle: "italic" }}>—</span>
```

```
        )}
      </div>
    </div>
  );
}
/* ─────────────────────────────────────────────
   CUSTOM SCATTER DOT
──────────────────────────────────────────────── */
const CustomDot = (props) => {
  const { cx, cy, payload } = props;
  if (payload?.isPareto) {
    const c = PARETO COLORS[payload.paretoIdx] || "#888";
    return (
      <g>
        <circle cx={cx} cy={cy} r={14} fill={c} opacity={0.15} />
        <circle cx={cx} cy={cy} r={7}  fill={c} stroke="#fff" strokeWidth={1.5} />
      </g>
    );
  }
  return <circle cx={cx} cy={cy} r={3.5} fill="#CBD5E1" opacity={0.6} />;
};
/* ─────────────────────────────────────────────
   ROOT APP
──────────────────────────────────────────────── */
export default function App() {
  const [epsA,      setEpsA]      = useState(5.0);
  const [epsT,      setEpsT]      = useState(3.0);
  const [dealValue, setDealValue] = useState(500);
  const [synergy,   setSynergy]   = useState(50);
  const [result,     setResult]     = useState(null);
  const [strategies, setStrategies] = useState([]);
  const [computing,  setComputing]  = useState(false);
  const [aiLoading,  setAiLoading]  = useState(false);
  const [error,      setError]      = useState("");
```

```
  const [ran,        setRan]        = useState(false);
  const isValid = epsA > 0 && epsT > 0 && dealValue > 0;
  const simulate = useCallback(async () => {
    if (!isValid) return;
    setError(""); setComputing(true); setResult(null); setStrategies([]);
    // Artificial compute delay for UX
    await new Promise(r => setTimeout(r, 1200));
    const { all, pareto } = runOptimization(epsA, epsT, dealValue, synergy);
    setComputing(false);
    if (pareto.length === 0) {
      setError("No feasible scenarios found. Try increasing synergy or adjusting EPS inputs.");
      return;
    }
    setResult({ all, pareto });
    setRan(true);
    setAiLoading(true);
    try {
      const insights = await fetchStrategies(epsA, pareto);
      setStrategies(insights);
    } catch {
      setStrategies([]);
    }
    setAiLoading(false);
  }, [epsA, epsT, dealValue, synergy, isValid]);
  const chartData = result ? [
    ...result.all.map(s => ({ ...s, isPareto: false })),
    ...result.pareto.map((s, i) => ({ ...s, isPareto: true, paretoIdx: i })),
  ] : [];
  const bestEPS  = result?.pareto?.[0]?.eps;
  const bestSens = result?.pareto?.[result.pareto.length - 1]?.sensitivity;
  return (
    <>
      <style>{STYLES}</style>
      <div className="maq-root">
        {/* ── HEADER ── */}
```

```
        <header className="maq-header">
          <div className="maq-logo">
            <div className="maq-logo-mark">
              <svg viewBox="0 0 16 16" fill="none">
                <path d="M3 13 L8 3 L13 13" stroke="white" strokeWidth="1.5" strokeLinecap="round"
strokeLinejoin="round"/>
                <path d="M5 9.5 H11" stroke="white" strokeWidth="1.5" strokeLinecap="round"/>
              </svg>
            </div>
            <div>
              <div className="maq-logo-title">M&A Quant Optimizer</div>
              <div className="maq-logo-sub">Pareto-Efficient Deal Structuring</div>
            </div>
          </div>
          <div className="maq-header-meta">
            <span className="maq-badge">
              <span className="maq-badge-dot" />
              Deterministic Engine v2.0
            </span>
            <span className="maq-badge">IEEE Research · Patent Pending</span>
          </div>
        </header>
        {/* ── SIDEBAR ── */}
        <aside className="maq-sidebar">
          <div>
            <div className="maq-section-label">Deal Parameters</div>
            <InputField label="Acquirer EPS" hint="Base EPS ($/share)" prefix="$" value={epsA}
setter={setEpsA} min={0.1} max={500} step={0.1} />
            <InputField label="Target EPS" hint="Pre-deal EPS" prefix="$" value={epsT}
setter={setEpsT} min={0.1} max={500} step={0.1} />
            <InputField label="Deal Value" hint="Total ($M)" prefix="$" value={dealValue}
setter={setDealValue} min={1} max={999999} step={1} />
          </div>
          <div>
            <div className="maq-slider-wrap">
              <div className="maq-slider-header">
```

```
                <span className="maq-slider-label">Base Synergy</span>
                <span className="maq-slider-val">${synergy}M</span>
              </div>
              <input
                type="range" className="maq-slider"
                min={0} max={500} step={5} value={synergy}
                onChange={e => setSynergy(+e.target.value)}
              />
              <div className="maq-slider-
ticks"><span>$0</span><span>$250M</span><span>$500M</span></div>
            </div>
          </div>
          <button
            className={`maq-btn maq-btn-primary${computing ? " maq-btn-shimmer" : ""}`}
            onClick={simulate}
            disabled={!isValid || computing}
          >
            {computing ? "Computing Frontier…" : "Simulate Optimal Deals →"}
          </button>
          {error && (
            <div className="maq-error">
              <svg width="14" height="14" viewBox="0 0 14 14" fill="none" style={{ flexShrink: 0,
marginTop: 1 }}>
                <circle cx="7" cy="7" r="6" stroke="currentColor" strokeWidth="1.3"/>
                <path d="M7 4.5V7.5" stroke="currentColor" strokeWidth="1.3" strokeLinecap="round"/>
                <circle cx="7" cy="9.5" r="0.7" fill="currentColor"/>
              </svg>
              {error}
            </div>
          )}
          <div>
            <div className="maq-section-label">Hard Constraints</div>
            <div className="maq-constraints">
              {[
                ["C + S = 100%", "Financing completeness enforced"],
                ["EPS_post ≥ 0.90 × EPS_base", "Max 10% dilution floor"],
```

```
                ["Y ∈ {0, Base, 1.5×}", "Synergy realism tiers"],
              ].map(([formula, desc]) => (
                <div className="maq-constraint" key={formula}>
                  <div className="maq-constraint-icon">
                    <svg viewBox="0 0 9 9" fill="none">
                      <path d="M2 4.5L3.8 6.5L7 2.5" stroke="#059669" strokeWidth="1.2"
strokeLinecap="round" strokeLinejoin="round"/>
                    </svg>
                  </div>
                  <div>
                    <strong>{formula}</strong><br />{desc}
                  </div>
                </div>
              ))}
            </div>
          </div>
        </aside>
        {/* ── MAIN ── */}
        <main className="maq-main">
          {/* Stats bar */}
          <div className="maq-stats">
            <StatCard label="Total Scenarios" value={result ? result.all.length : "—"} sub="Grid
search space" skeleton={computing} />
            <StatCard label="Pareto Optimal" value={result ? result.pareto.length : "—"} sub="Non-
dominated" colored={!!result} skeleton={computing} />
            <StatCard label="Best EPS" value={bestEPS ? `$${bestEPS.toFixed(3)}` : "—"} sub="Maximum
return" skeleton={computing} />
            <StatCard label="Min Sensitivity" value={bestSens != null ? bestSens.toFixed(3) : "—"}
sub="Lowest risk score" skeleton={computing} />
          </div>
          {/* Chart */}
          <div className="maq-chart-card" style={{ position: "relative" }}>
            {computing && (
              <div className="maq-compute-overlay">
                <div className="maq-compute-spinner" />
                <div className="maq-compute-label">Running grid search…</div>
                <div className="maq-compute-sub">Extracting Pareto frontier</div>
```

```
              </div>
            )}
            <div className="maq-chart-header">
              <div>
                <div className="maq-chart-title">Pareto Efficiency Frontier</div>
                <div className="maq-chart-subtitle">Risk-Return optimization landscape across all deal
configurations</div>
              </div>
              {ran && result && (
                <div className="maq-legend">
                  {result.pareto.slice(0, 3).map((s, i) => (
                    <div className="maq-legend-item" key={i}>
                      <div className="maq-legend-dot" style={{ background: PARETO_COLORS[i] }} />
                      {s.label}
                    </div>
                  ))}
                  <div className="maq-legend-item">
                    <div className="maq-legend-dot" style={{ background: "#CBD5E1" }} />
                    Dominated
                  </div>
                </div>
              )}
            </div>
            {!ran && !computing ? (
              <div className="maq-empty">
                <div className="maq-empty-icon">
                  <svg width="24" height="24" viewBox="0 0 24 24" fill="none">
                    <path d="M4 20 L9 14 L13 17 L20 9" stroke="#94A3B8" strokeWidth="1.5"
strokeLinecap="round" strokeLinejoin="round"/>
                    <circle cx="9" cy="14" r="1.5" fill="#94A3B8"/>
                    <circle cx="13" cy="17" r="1.5" fill="#94A3B8"/>
                    <circle cx="20" cy="9" r="1.5" fill="#94A3B8"/>
                  </svg>
                </div>
                <div className="maq-empty-title">No simulation yet</div>
```

```
                <div className="maq-empty-sub">Configure deal parameters and click Simulate to
generate the Pareto frontier.</div>

              </div>

            ) : (

              <ResponsiveContainer width="100%" height={300}>

                <ScatterChart margin={{ top: 10, right: 30, bottom: 30, left: 10 }}>

                  <CartesianGrid strokeDasharray="4 4" stroke="rgba(15,23,42,0.06)" />

                  <XAxis

                    dataKey="sensitivity" type="number" domain={["auto", "auto"]}

                    tick={{ fontSize: 11, fill: "#94A3B8", fontFamily: "'DM Mono', monospace" }}

                    label={{ value: "Sensitivity  (Risk →)", position: "insideBottom", offset: -16,
fontSize: 12, fill: "#94A3B8" }}

                    tickLine={false} axisLine={{ stroke: "rgba(15,23,42,0.1)" }}

                  />

                  <YAxis

                    dataKey="eps" type="number" domain={["auto", "auto"]}

                    tick={{ fontSize: 11, fill: "#94A3B8", fontFamily: "'DM Mono', monospace" }}

                    tickFormatter={v => `$${v.toFixed(1)}`}

                    label={{ value: "EPS (Return ↑)", angle: -90, position: "insideLeft", offset: 20,
fontSize: 12, fill: "#94A3B8" }}

                    tickLine={false} axisLine={false}

                    width={56}

                  />

                  <Tooltip content={<CustomTooltip />} cursor={{ stroke: "rgba(15,23,42,0.1)",
strokeDasharray: "4 4" }} />

                  <Scatter data={chartData} shape={<CustomDot />} />

                </ScatterChart>

              </ResponsiveContainer>

            )}

          </div>

        </main>

        {/* ── STRATEGY CARDS ── */}

        {ran && result?.pareto?.length > 0 && (

          <section className="maq-cards-section" style={{ gridColumn: "2", gridRow: "3" }}>

            <div className="maq-cards-header">

              <div className="maq-cards-title">Top Pareto Configurations · Strategic Analysis</div>

              <div className="maq-ai-badge">
```

```
                <svg width="9" height="9" viewBox="0 0 9 9" fill="none">
                  <circle cx="4.5" cy="4.5" r="3.5" stroke="currentColor" strokeWidth="1"/>
                  <path d="M4.5 2.5V4.5L5.5 5.5" stroke="currentColor" strokeWidth="1"
strokeLinecap="round"/>
                </svg>
                AI-Augmented
              </div>
            </div>
            <div className="maq-cards-grid">
              {result.pareto.slice(0, 3).map((s, i) => {
                const insight = strategies.find(st => st.type === s.label)?.insight || null;
                return <ScenarioCard key={i} scenario={s} insight={insight} aiLoading={aiLoading}
idx={i} />;
              })}
            </div>
          </section>
        )}
        {/* ── FOOTER ── */}
        <footer className="maq-footer">
          <span>Constrained Multi-Objective Optimization · EPS Maximization · Pareto Frontier
Extraction</span>
          <div className="maq-footer-pills">
            <span className="maq-footer-pill">Grid Search Δ10%</span>
            <span className="maq-footer-pill">3 Synergy Tiers</span>
            <span className="maq-footer-pill">Deterministic Engine</span>
          </div>
        </footer>
      </div>
    </>
  );
}
```